时序大模型调研

1. 调研目标

- 跑通流程

  确保从数据预处理、模型训练到评估的完整流程顺畅运行。
- 超参数调优脚本开发

  编写自动化脚本，以系统地调整各个模型的超参数，提升模型性能
- 对比多个时序模型的效果

2. 数据

特征列	解释
Timestamp	时间戳，通常用于表示数据记录的时间和日期，按照特定格式排列（如年 - 月 - 日时：分：秒）。
Value_NE5	这可能是一个特定的观测值或传感器读数，具体含义需要根据数据收集的上下文来理解。例如，在某些环境监测系统中，可能代表某种气体浓度或特定成分的含量。
Value_NE7	同上，是另一个需要根据具体情况确定含义的传感器数值或测量结果。可能与 Value_NE5 隶属于同一类测量或不同类别的相关测量。
Hr [%Hr]	通常指相对湿度，即空气中实际水汽含量与相同温度下饱和水汽含量的比值（以百分比表示）。
RainDur [min]	降雨持续时间，即连续降雨所持续的时间，单位为分钟。
StrGlo [W/m2]	总辐射强度，是指单位时间内进入单位面积的辐射能，包括直接辐射和散射辐射的总和，单位为瓦特每平方米。
T [°C]	温度，采用摄氏度作为单位，表示物体或环境的冷热程度。
WD [°]	风向，一般以角度表示，从正北方向开始顺时针旋转，到 360°（其中 0°和 360°均表示正北方向）。
WVs [m/s]	水平风速，表示空气在水平方向上的流动速度，单位为米每秒。
WVv [m/s]	垂直风速，表示空气在垂直方向上的流动速度，单位为米每秒，正值向上，负值向下。

p [hPa]	气压，采用百帕作为单位，表示大气压强，即大气压力。

zurich_electricity_consumption_tt.csv
20.75MB

3. 模型训练

3.1 Chronos

1. 训练代码：

```
# 创建 TimeSeriesPredictor1
predictor = TimeSeriesPredictor(
    prediction_length=prediction_length,#预测长度
    target="Value_NE5",#目标特征
    known_covariates_names=['WVs [m/s]', 'WVv [m/s]','p [hPa]'],#未来协变量
    #chronos将除未来协变量外的所有特征作为过去协变量，不需要额外设置
).fit(
    data_train,
    hyperparameters={
        "Chronos": [
            # Chronos-Bolt (Small) combined with CatBoost on covariates
            {
                "model_path": "bolt_small",
                "covariate_regressor": "CAT",
                "fine_tune":True,#是否进行微调
                "target_scaler": "standard",
                "context_length": context_length,#上下文长度
                "ag_args": {"name_suffix": "WithRegressor"},
            },
        ],
    },
    enable_ensemble=False,
    # time_limit=60, #模型训练过程的最大时间限制（秒），默认没有时间限制
)
```

2. 预测代码：

```
# 获取当前时间点的位置
current_pos = len(self.data_test) - 1  # 定位到当前的最后一个数据点

# 提取上下文数据
```

```
context_length = context_length  # 上下文窗口的长度
context_start = current_pos - context_length  # 从当前时间点往前取context_length 长度的上下文数据
context_data = data_test.iloc[context_start:current_pos + 1]  # 上下文数据，包括当前时间点的数据

# 创建 TimeSeriesDataFrame 输入
context_ts = TimeSeriesDataFrame(context_data)

# 获取协变量数据，确保协变量数据与预测时间点对齐
feature_events_ts = TimeSeriesDataFrame(feature_events)

# 使用模型进行预测
forecast_pre = predictor.predict(context_ts, known_covariates=feature_events_ts)
```

3.2 TFTModel

1. 训练代码：

```
model = TFTModel(
    input_chunk_length=context_length,
    output_chunk_length=predict_length,
    n_epochs=5,
)

model.fit(target, past_covariates=past_cov, future_covariates=future_cov)
```

2. 预测代码：

```
target = test_target[:context_length+i+1]
past_cov = test_past_cov[:context_length+i+1]
future_cov = test_future_cov[:context_length+i+2]

# Make prediction for the next point (forecast horizon = 1)
pred = model.predict(predict_length,
                     series=target,
                     past_covariates=past_cov,
                     future_covariates=future_cov,
                     verbose=False,
                     num_samples=200
)
```

3.3 TiDEModel

1. 训练代码：

```
model = TiDEModel(
    input_chunk_length=context_length,  # 输入序列的长度
    output_chunk_length=predict_length,  # 输出序列的长度
    n_epochs=20,
)

# 模型训练
model.fit(
    series=target,  # 你的目标序列
    past_covariates=past_cov,
    future_covariates=future_cov,
    verbose=True,  # 默认为 False，设置为 True 显示训练进度
)
```

2. 预测代码：

```
target = test_target[:context_length+i+1]
past_cov = test_past_cov[:context_length+i+1]
future_cov = test_future_cov[:context_length+i+2]

# Make prediction for the next point (forecast horizon = 1)
pred = model.predict(predict_length,
                     series=target,
                     past_covariates=past_cov,
                     future_covariates=future_cov,
                     verbose=False,
                     num_samples=200
)
```

3.4 DLinear

1. 训练代码：

```
model = DLinearModel(
    input_chunk_length=context_length,
    output_chunk_length=predict_length,
    n_epochs=20,
)
```

```
model.fit(target, past_covariates=past_cov, future_covariates=future_cov)
```

2. 预测代码：

```
target = test_target[:context_length+i+1]
past_cov = test_past_cov[:context_length+i+1]
future_cov = test_future_cov[:context_length+i+2]

# Make prediction for the next point (forecast horizon = 1)
pred = model.predict(predict_length,
                     series=target,
                     past_covariates=past_cov,
                     future_covariates=future_cov,
                     verbose=False
)
```

3.5 TSMixer

1. 训练代码：

```
model = TSMixerModel(
    input_chunk_length=context_length,  # 输入序列的长度
    output_chunk_length=predict_length,  # 输出序列的长度
    n_epochs=20,  # 训练的最大 epoch 数
)

# 模型训练
model.fit(
    series=target,  # 你的目标序列
    past_covariates=past_cov,
    future_covariates=future_cov,
    verbose=True,  # 默认为 False，设置为 True 显示训练进度
)
```

2. 预测代码：

```
target = test_target[:context_length+i+1]
past_cov = test_past_cov[:context_length+i+1]
future_cov = test_future_cov[:context_length+i+2]

# Make prediction for the next point (forecast horizon = 1)
```

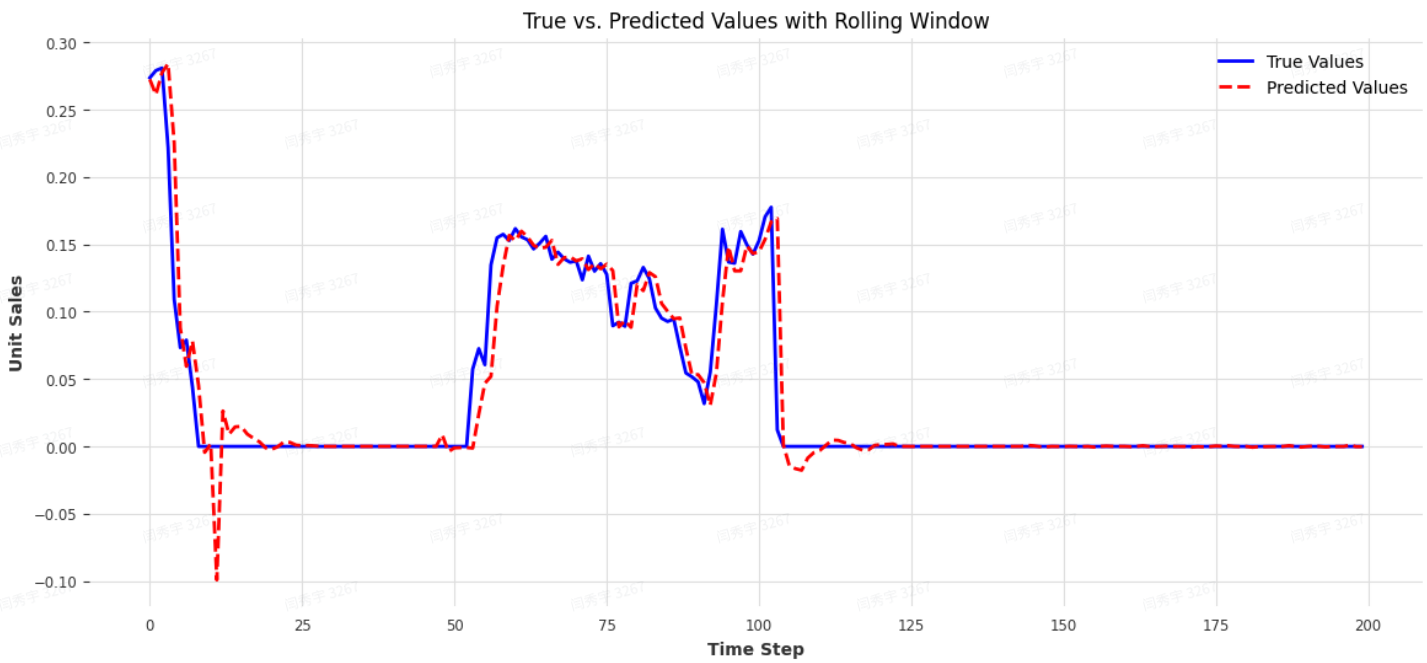
```
pred = model.predict(predict_length,
                     series=target,
                     past_covariates=past_cov,
                     future_covariates=future_cov,
                     verbose=False,
                     num_samples=200
)
```

4. 模型对比

4.1 上下文长度=20

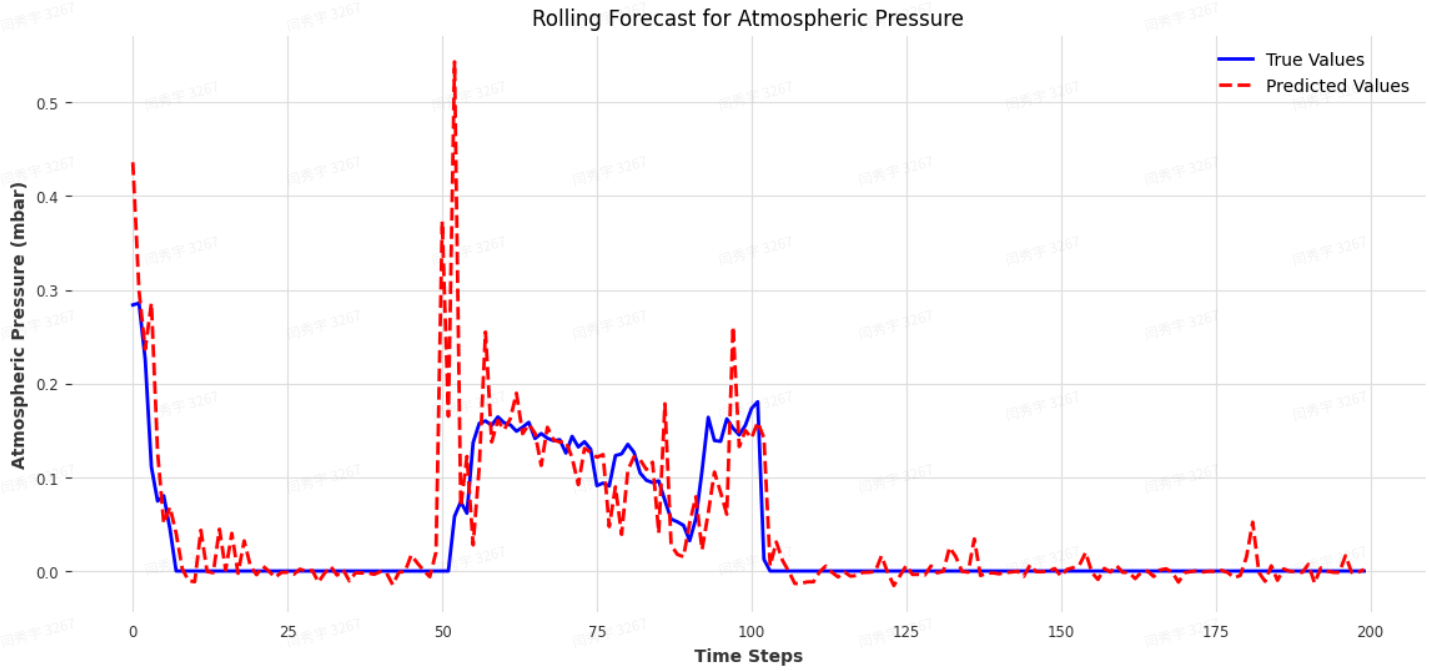
1. Choros

```
Mean Squared Error (MSE): 0.0004
R-squared (R2): 0.8941
```



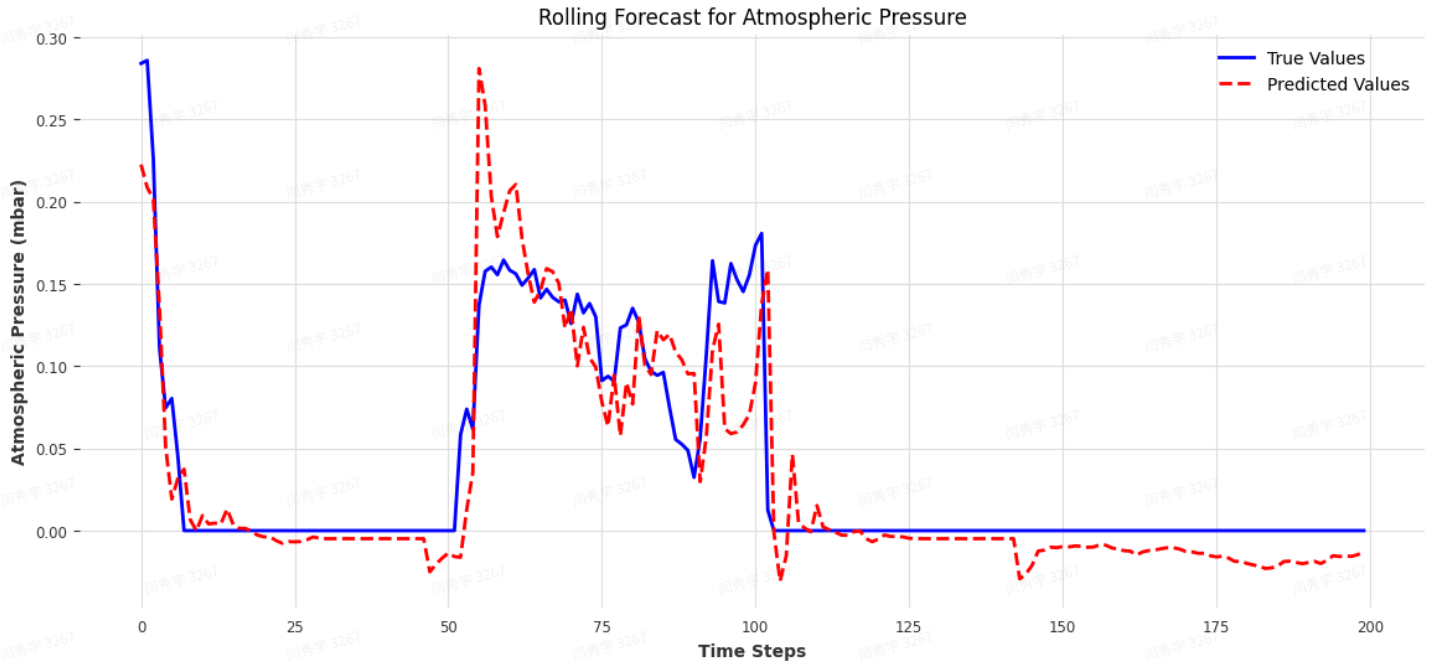
2. TFTModel

```
R² Score: 0.2448803926466283
Mean Squared Error (MSE): 0.003028064164573888
```



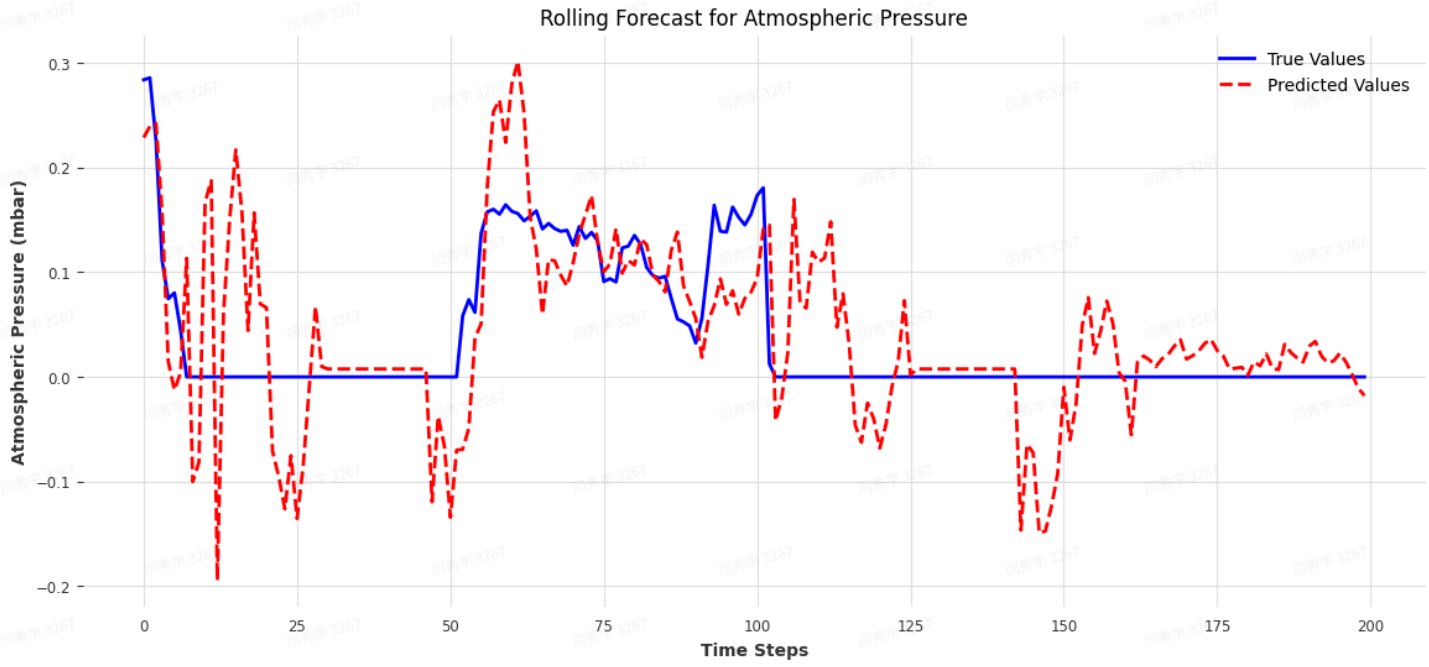
3. TiDEModel

```
R² Score: 0.7582812776320639
Mean Squared Error (MSE): 0.0009693031328829047
```



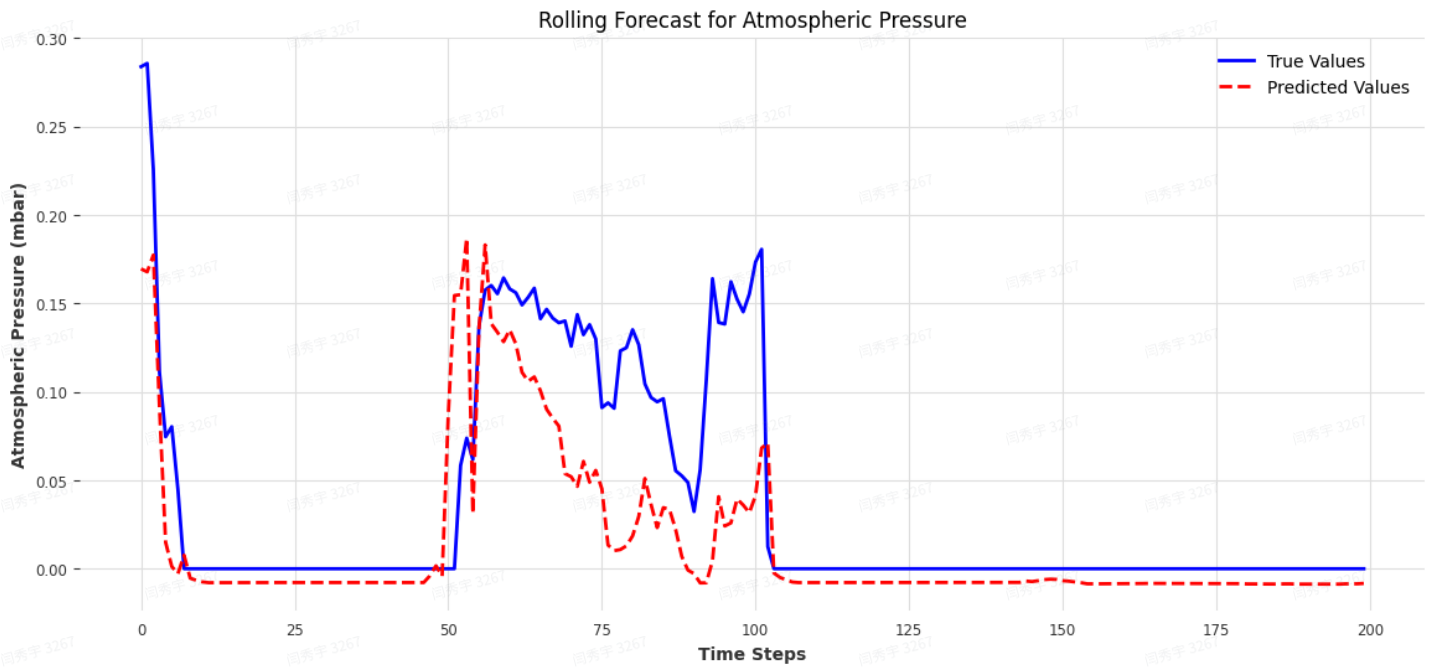
4. DLinear

```
R² Score: -0.10199370898265592
Mean Squared Error (MSE): 0.004419045178090153
```



5. TSMixer

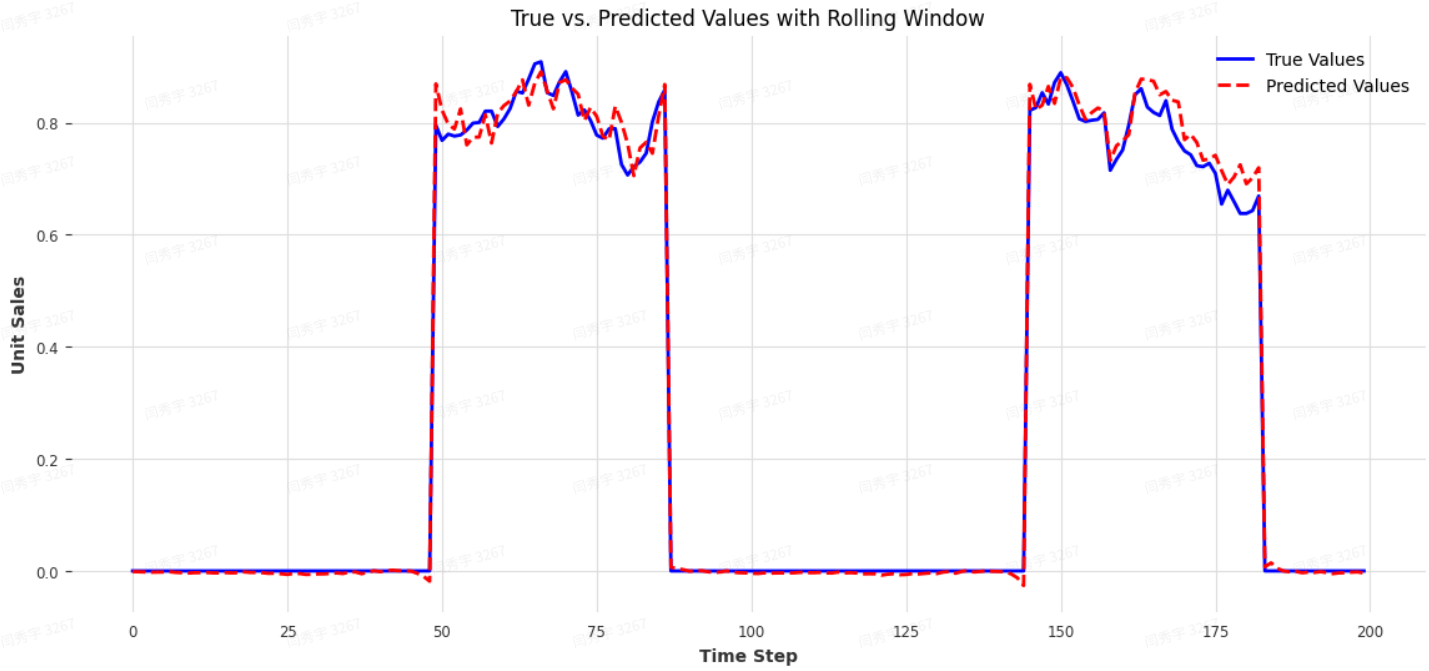
```
R² Score: 0.485207127299241
Mean Squared Error (MSE): 0.0020643429660987937
```



4.2 上下文长度=512

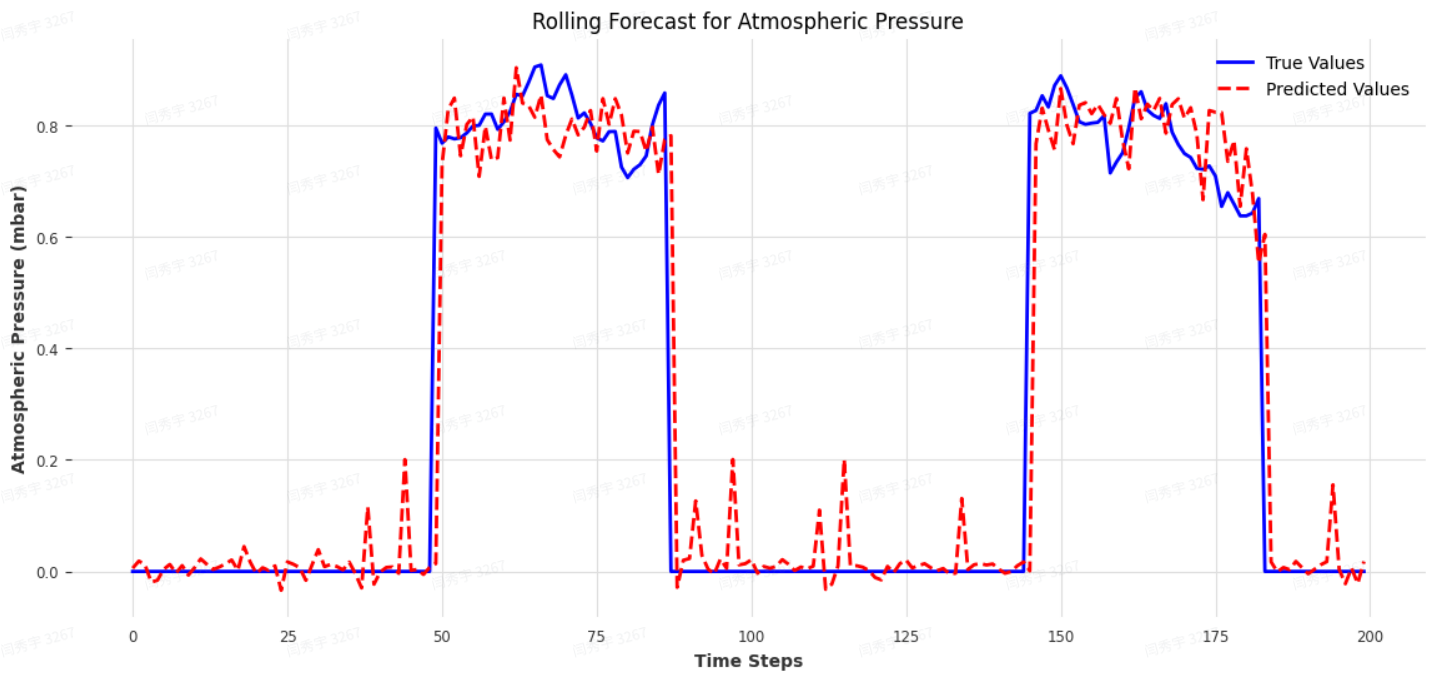
1. Choros

```
Mean Squared Error (MSE): 0.0005
R-squared (R2): 0.9968
```



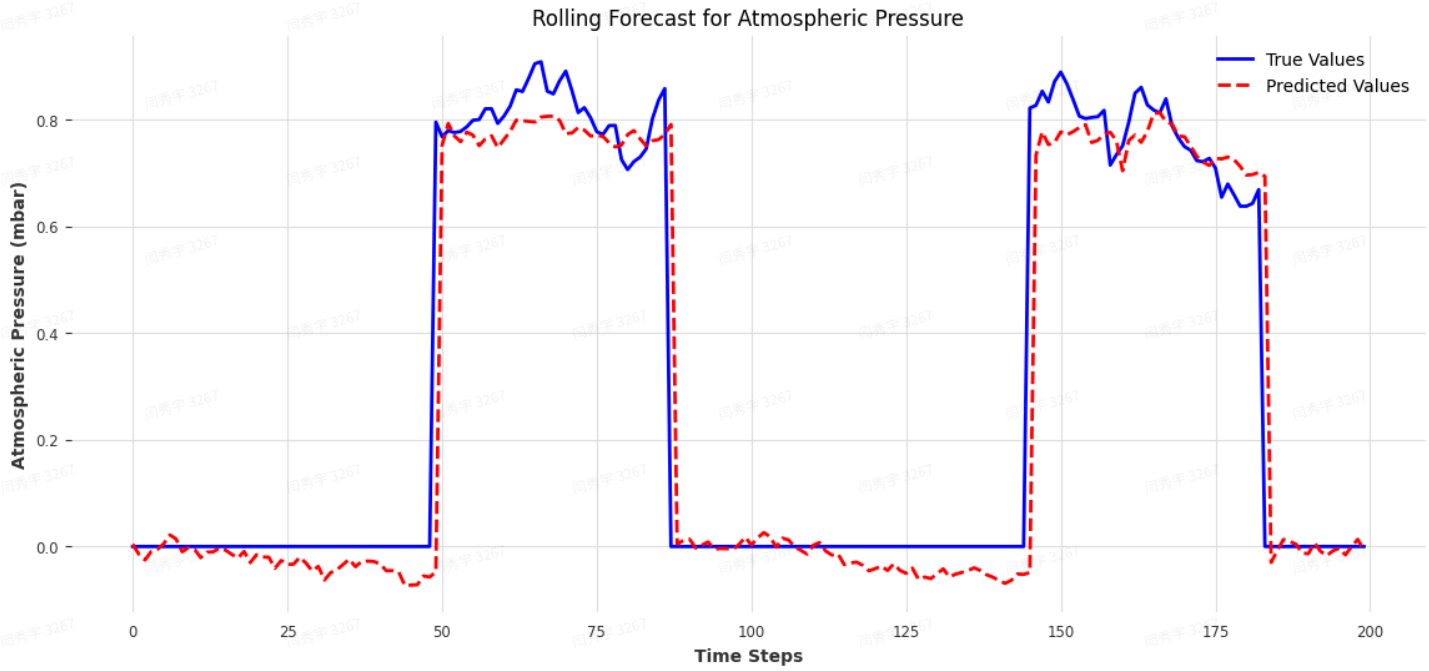
2. TFTModel

```
R² Score: 0.9051248349236989
Mean Squared Error (MSE): 0.014139171780456766
```



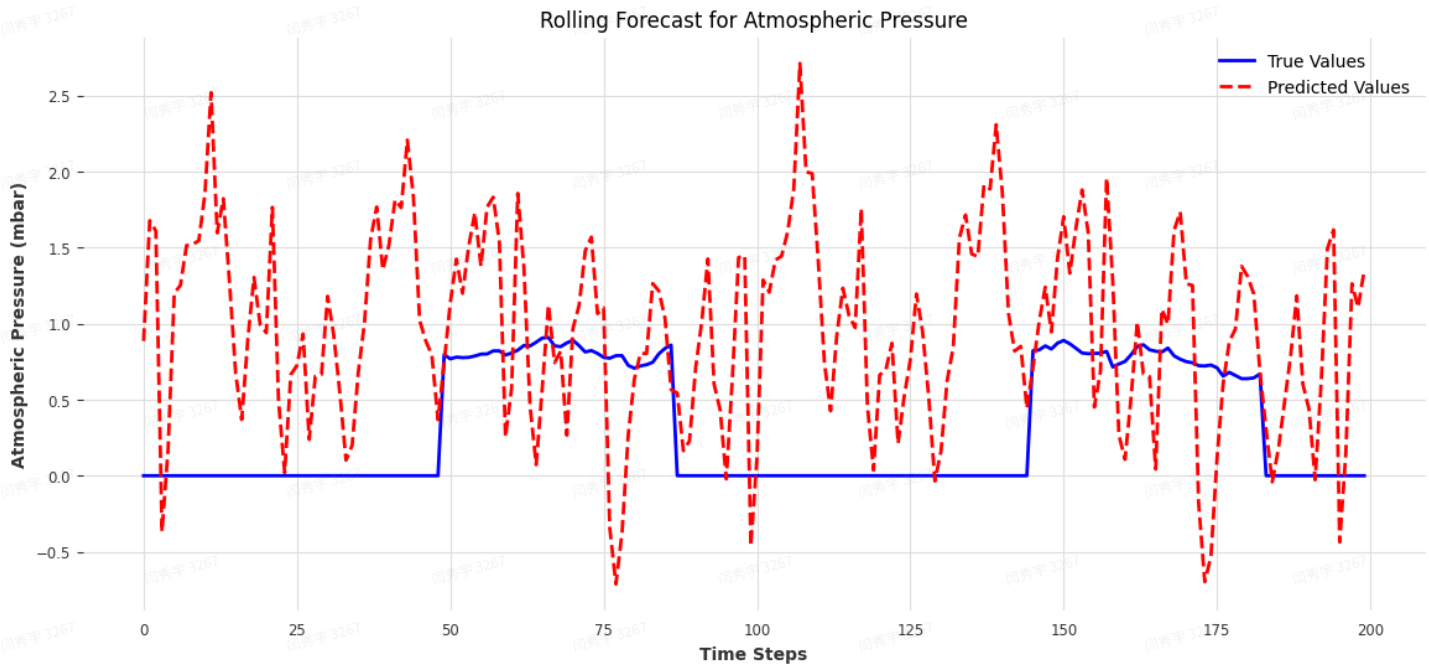
3. TiDEModel

```
R² Score: 0.9012005977345097
Mean Squared Error (MSE): 0.014723997785034257
```



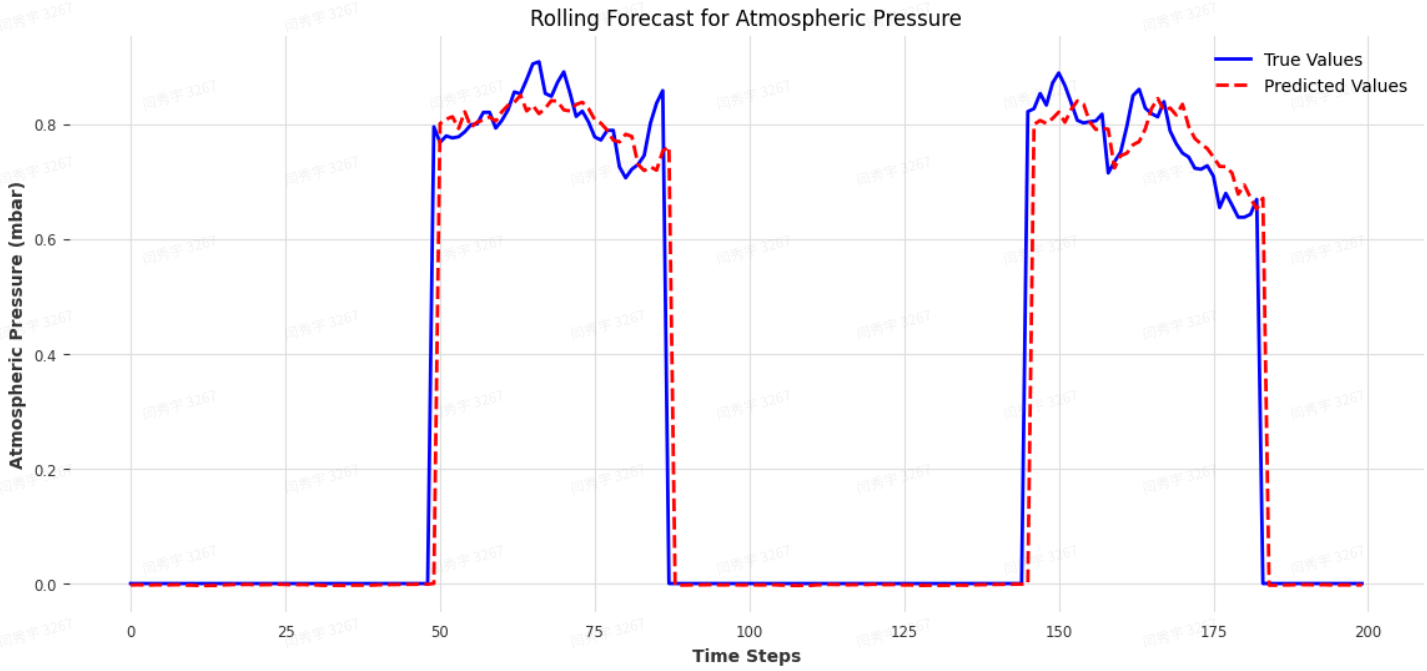
4. DLinear

```
R² Score: -5.773230013416268
Mean Squared Error (MSE): 1.0094091809086079
```

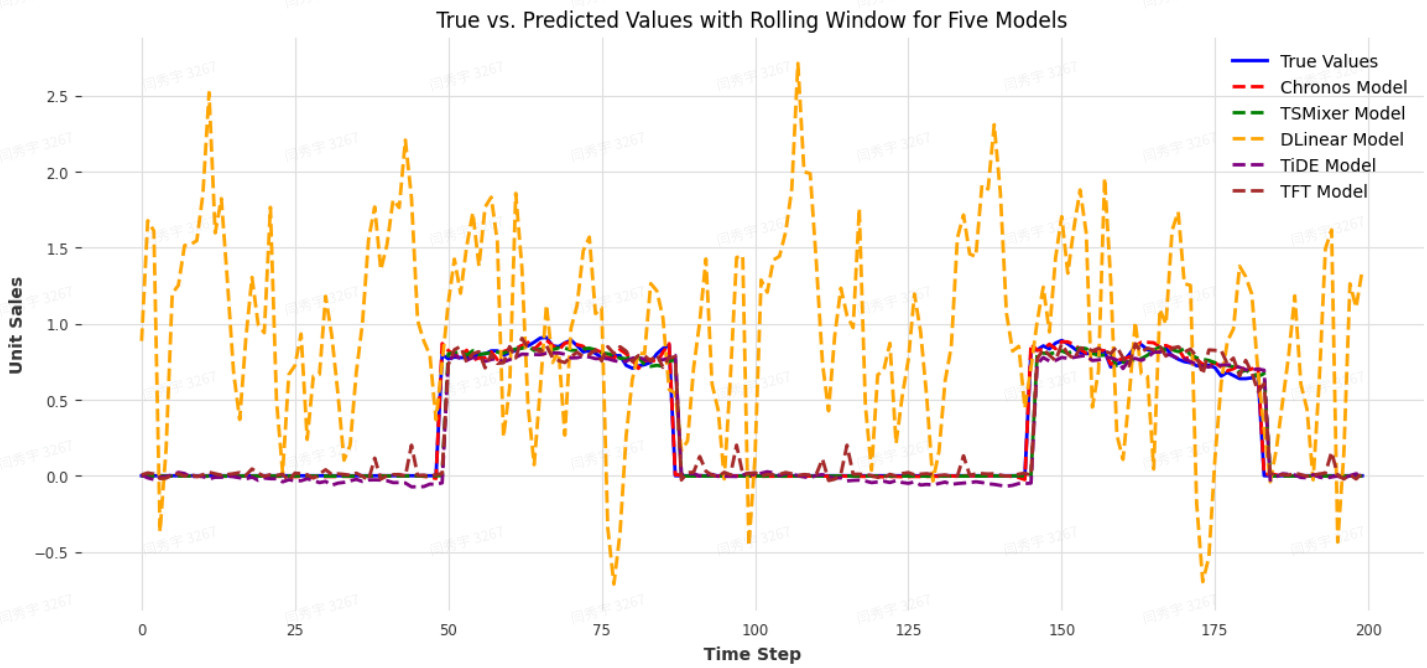


5. TSMixer

```
R² Score: 0.9162153757326345
Mean Squared Error (MSE): 0.012486357142299412
```



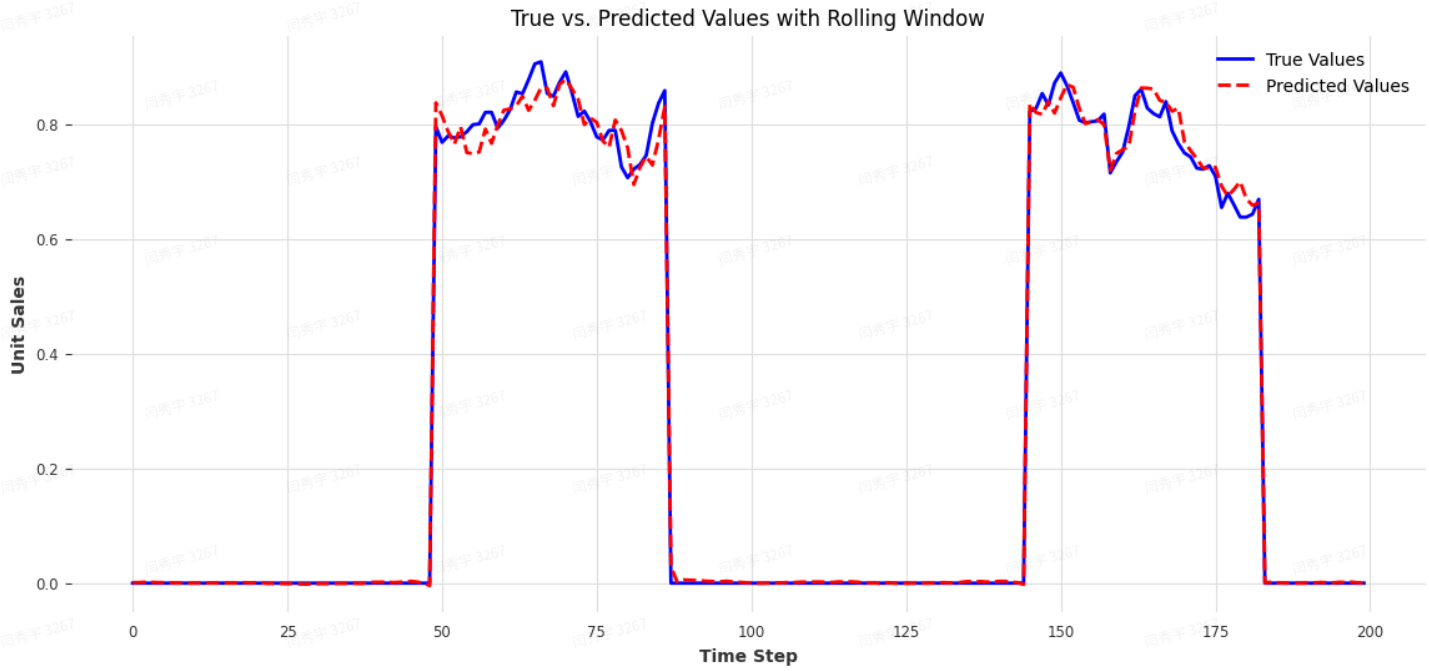
6. 总结



4.3 上下文长度=3800

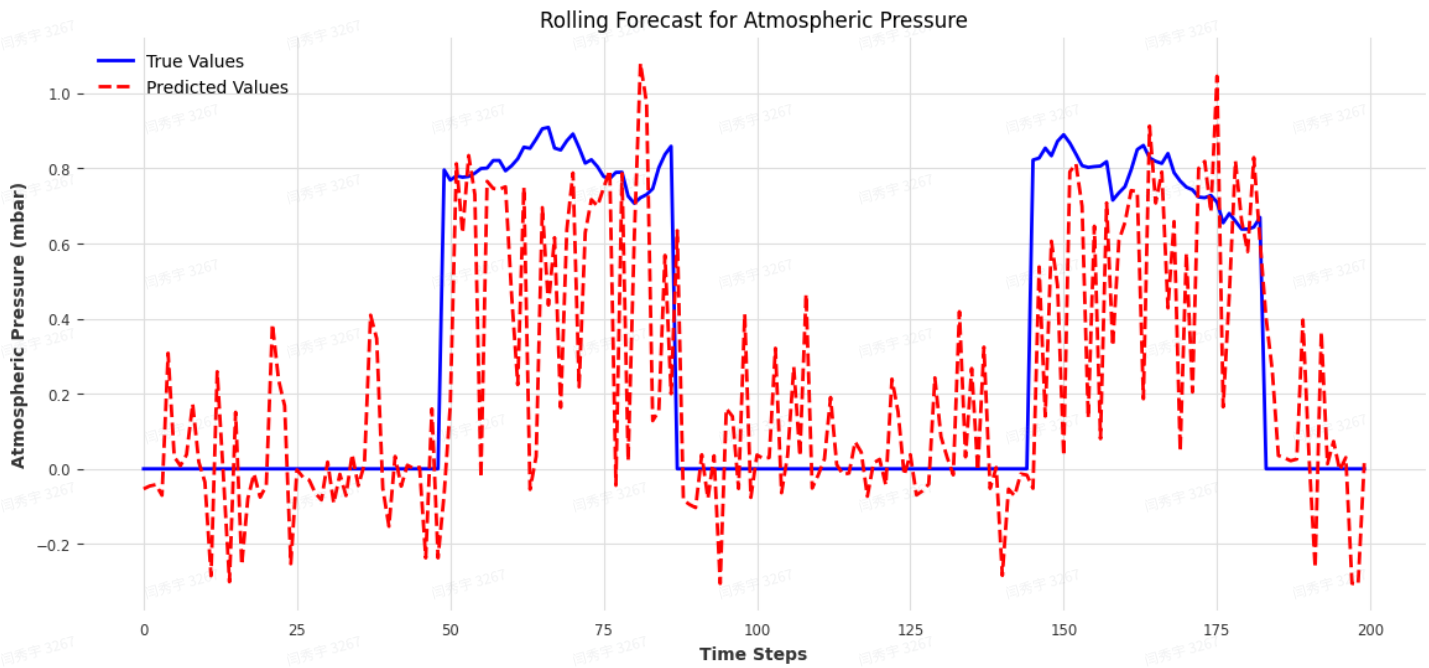
1. Choros

```
Mean Squared Error (MSE): 0.0004
R-squared (R2): 0.9975
```



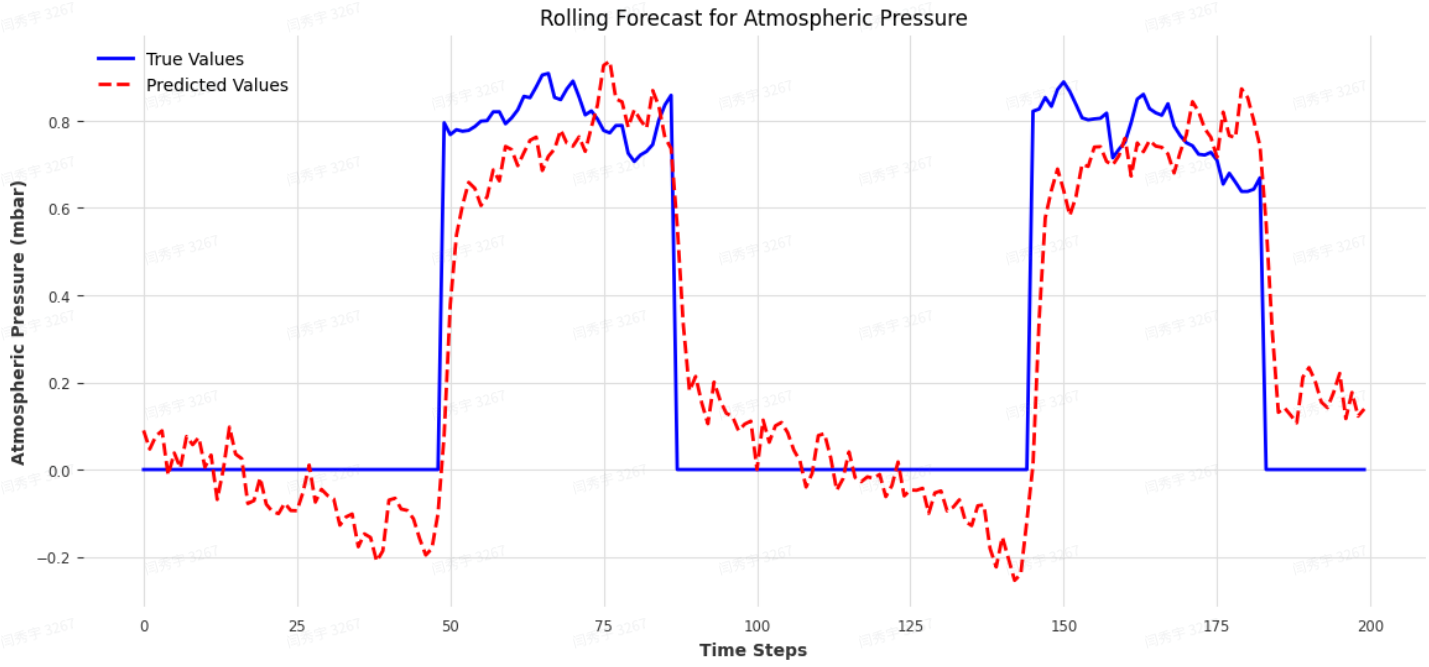
2. TFTModel

```
R² Score: 0.422237180733428
Mean Squared Error (MSE): 0.08610354188476256
```



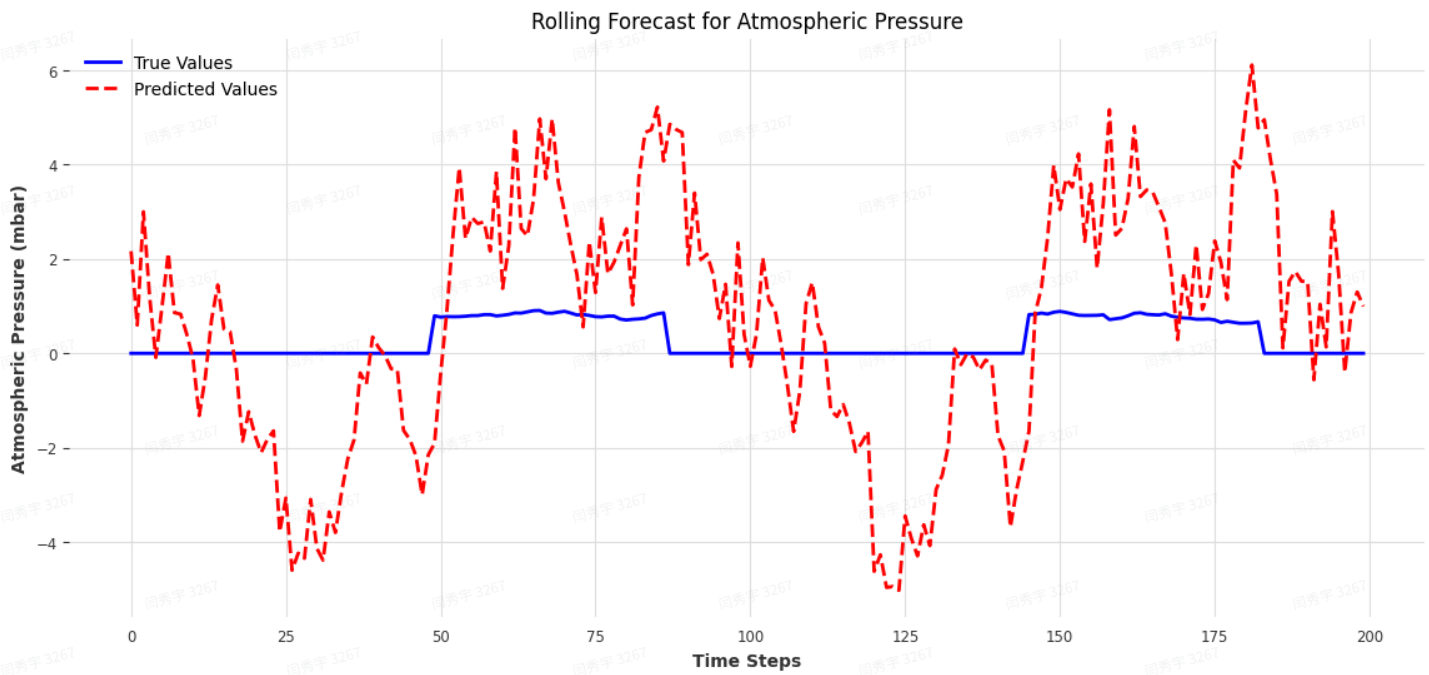
3. TiDEModel

```
R² Score: 0.8249124408324462
Mean Squared Error (MSE): 0.02609316224851881
```



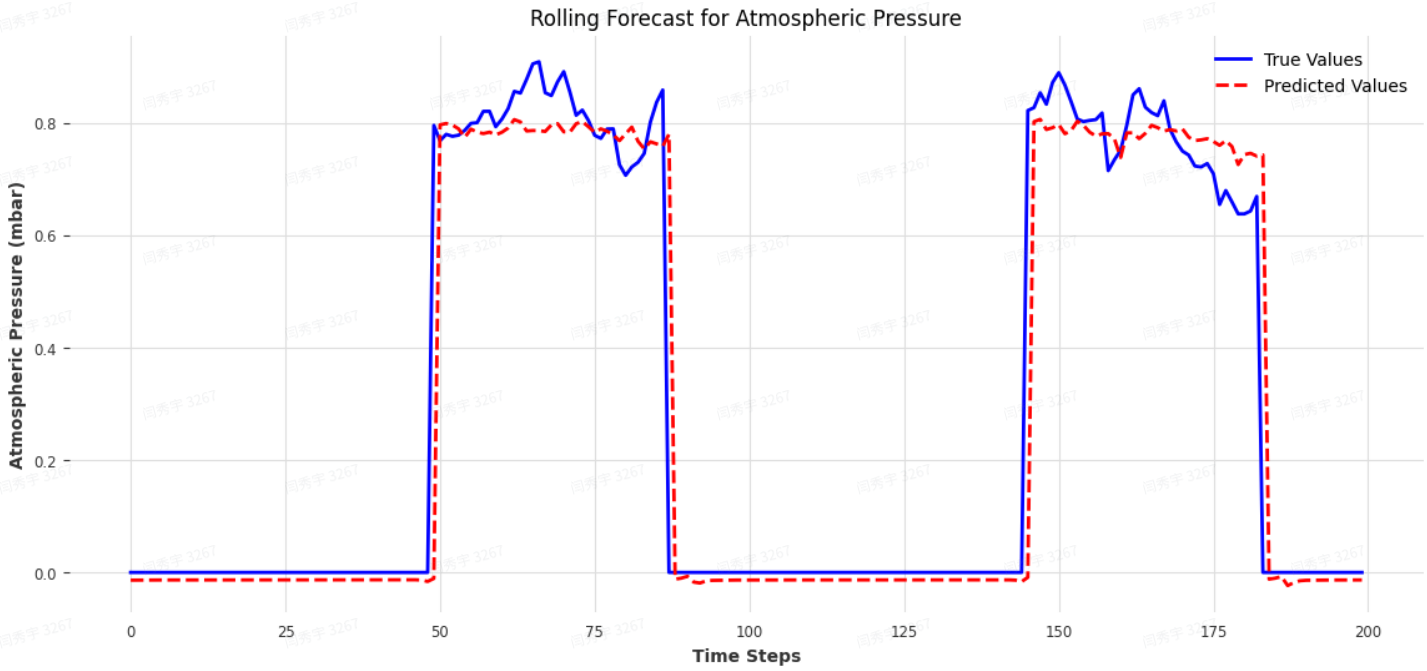
4. DLinear

```
R² Score: -37.943326881807096
Mean Squared Error (MSE): 5.803693601392122
```



5. TSMixer

```
R² Score: 0.9071762047281855
Mean Squared Error (MSE): 0.013833457740037938
```



5. 总结

	context_length=20		context_length=512		context_length=3800	
	mse	R2	mse	R2	mse	R2
chronos	0.0004	0.0941				
TFTModel	0.0030	0.2449				
TiDEModel	0.0010	0.7583				
DLinear	0.0044	-0.1020				
TSMixer	0.0021	0.4852				